

КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМ. ІГОРЯ СІКОРСЬКОГО

ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №1

на тему:

«Вибір та реалізація базових фреймворків та бібліотек»

Виконали:

студентки групи ФІ-62 мн

Дегтярьова Марія

Тимошенко Марія

Зміст

1	Дослідження принципів реалізації криптографічних механізмів	2
2	Порівняння криптографічних бібліотек	6
3	Практичне використання криптографічних бібліотек	10
4	Вибір криптографічної бібліотеки для різних ІТ-систем	14
5	Індивідуальне завдання	16
	Висновки	17
	Перелік використаних джерел	18

Мета роботи: дослідження сучасних програмних бібліотек реалізації криптографічних механізмів, критеріїв їх вибору для різних програмно-апаратних платформ та отримання практичних навичок коректного використання криптографічних API.

1 Дослідження принципів реалізації криптографічних механізмів

1.1 Чому при розробці прикладного програмного забезпечення не рекомендується самостійно реалізовувати стандартні криптографічні алгоритми?

“Custom Algorithms → Don’t do this”. [1]

Не рекомендується самостійно реалізовувати стандартні криптографічні алгоритми, оскільки навіть невелика помилка в їх реалізації може призвести до появи вразливості та зниження рівня захисту даних. Криптографічні бібліотеки вже містять готові реалізації поширених алгоритмів, які були протестовані та перевірені.

Крім того, відомі криптографічні бібліотеки регулярно оновлюються для виправлення виявлених помилок і вразливостей. Тому замість створення власної реалізації стандартного алгоритму доцільно використовувати надійні та широко перевірені криптографічні бібліотеки, як це рекомендує OWASP.

1.2 Які основні групи криптографічних примітивів та допоміжних функцій зазвичай надають сучасні криптографічні бібліотеки?

Сучасні криптографічні бібліотеки зазвичай містять такі основні групи криптографічних примітивів: [4]:

- односторонні хеш-функції — обчислюють хеш-значення повідомлення;
- симетрична криптографія — забезпечує шифрування та розшифрування за допомогою одного ключа;
- криптографія з відкритим ключем — використовує різні ключі для шифрування та розшифрування;
- цифрові підписи — використовуються для підтвердження автентичності автора та цілісності повідомлення;
- криптографічно стійкі генератори псевдовипадкових чисел (CSPRNG) — використовуються для отримання непередбачуваних випадкових значень, зокрема криптографічних ключів та токенів.

Крім наведених примітивів, сучасні криптографічні бібліотеки можуть надавати й додаткові механізми, зокрема функції узгодження та отримання ключів, MAC, а також засоби роботи із сертифікатами та ключами.

1.3 У чому полягає різниця між криптографічним примітивом та криптографічним протоколом? Навести приклади.

Криптографічний примітив — низькорівневий криптографічний алгоритм, який використовується як базовий будівельний блок для криптографічних алгоритмів вищого рівня [2].

Криптографічний протокол — послідовність кроків та обмінів повідомленнями між кількома сторонами з метою досягнення певної цілі безпеки [3].

Таким чином, основна відмінність полягає в тому, що криптографічний примітив є окремим алгоритмічним компонентом, тоді як криптографічний протокол описує взаємодію між сторонами та порядок використання криптографічних примітивів для забезпечення необхідних властивостей безпеки.

Базовими криптографічними примітивами є односторонні хеш-функції, наприклад SHA-256, симетричні криптографічні алгоритми, наприклад AES, асиметричні криптографічні алгоритми, наприклад RSA, а також алгоритми цифрового підпису. [4] Прикладами криптографічних протоколів є Needham–Schroeder, Otway–Rees та Kerberos. [3]

1.4 Що таке криптографічно стійкий генератор псевдовипадкових чисел (CSPRNG) та чому звичайні генератори псевдовипадкових чисел не повинні використовуватися для генерації криптографічних ключів?

Випадкові числа та рядки використовуються для виконання важливих з погляду безпеки операцій, зокрема для генерації криптографічних ключів, векторів ініціалізації (BI), ідентифікаторів сеансів, CSRF-токенів та токенів для скидання пароля. Тому важливо, щоб такі значення було складно передбачити або відтворити злоумиснику.

Звичайні генератори псевдовипадкових чисел (ГПЧ) призначені переважно для завдань, де криптографічна безпека не потрібна. Вони можуть бути швидкими та зручними для таких операцій, як перемішування даних або випадковий вибір елементів. Проте їхній алгоритм або внутрішній стан у деяких випадках може бути передбачений, тому використання таких генераторів для створення криптографічних ключів може призвести до компрометації ключа.

Криптографічно стійкий генератор псевдовипадкових чисел (CSPRNG) призначений для генерування непередбачуваних випадкових значень, які можуть використовуватися в операціях, що потребують високого рівня безпеки. На відміну від звичайних ГПЧ, CSPRNG використовує механізми, які забезпечують криптографічну стійкість отриманих

значень. Тому саме CSPRNG слід використовувати для генерації криптографічних ключів, токенів, ВІ та інших значень, що мають бути непередбачуваними. [1]

1.5 У чому полягає різниця між низькорівневими (low-level) та високорівневими (high-level) криптографічними API? Які ризики може створювати неправильне використання низькорівневого API?

Високорівневий API (High-level): створений за принципом «захисту від помилок» (*secure-by-default*) [8]. Він приховує складні налаштування й надає прості функції: розробник передає дані та ключ, а бібліотека сама генерує випадковий nonce/IV потрібної довжини, додає автентифікаційний тег і правильно обирає безпечний режим (наприклад, модуль **Fernet** у бібліотеці **cryptography**) [5].

Низькорівневий API (Low-level): дає прямий доступ до базових примітивів (наприклад, шар **hazmat**) [5, 6]. Тут розробник повинен вручну обирати розмір блоку, режим шифрування, спосіб доповнення (padding) і сам керувати векторами ініціалізації.

Ризики неправильного використання низькорівневого API [1, 7]:

- Повторне використання Nonce/IV: якщо в режимах на зразок AES-GCM або ChaCha20 зашифрувати два різні повідомлення з однаковим nonce на одному ключі, це призводить до витоку відкритого тексту та можливості підробки повідомлень;
- Вибір небезпечного режиму: розробник може помилково обрати застарілий режим на зразок ECB, який не приховує структуру даних;
- Відсутність перевірки цілісності: використання звичайного шифрування без автентифікації (без MAC або AEAD) дозволяє зловмиснику непомітно змінювати байти шифротексту;
- Атаки на доповнення (Padding Oracle): за неправильної ручної обробки помилок доповнення PKCS#7 у режимі CBC зловмисник може повністю розшифрувати трафік.

1.6 Що означає реалізація криптографічної операції з постійним часом виконання (constant-time implementation) та від яких атак вона повинна захищати?

Реалізація з постійним часом означає, що криптографічна операція виконується за однакову кількість тактів процесора незалежно від того, які саме секретні дані (ключі, паролі, відкритий текст) обробляються [7].

Щоб цього досягти, у коді уникають:

- умовних розгалужень (**if**, **switch**), які залежать від секретних бітів (бо процесор передбачає переходи з різною швидкістю);

- звернень до пам'яті за індексами, що залежать від секрету (щоб час операції не залежав від потрапляння в кеш-пам'ять процесора).

Захищає від:

- Часові атаки (Timing attacks): коли зломисник із високою точністю вимірює час відповіді системи. Наприклад, якщо порівнювати MAC звичайним циклом або оператором `==`, функція зупиняється на першому ж неправильному байті — це дозволяє атакуючому байт за байтом підібрати правильне значення;
- Атаки через процесорний кеш (Cache-timing attacks): захищає від витоку інформації про те, які саме комірки пам'яті чи таблиці підстановок завантажувалися в кеш під час обчислень.

1.7 Що таке FIPS 140-3 та в яких випадках наявність відповідної сертифікації криптографічного модуля може бути важливою?

FIPS 140-3 є офіційним стандартом безпеки США та Канади (розроблений NIST), який встановлює вимоги до апаратних і програмних криптографічних модулів [9, 10]. Він оцінює якість генерації ключів, захист від витоків, коректність реалізації алгоритмів та механізми самотестування модуля. Стандарт поділяється на 4 рівні безпеки (від базового програмного захисту Level 1 до серйозного фізичного захисту від розтину Level 4) [11].

Наявність сертифіката важлива, коли:

- Державні та оборонні контракти: у США/НАТО використання модулів без сертифікації FIPS заборонено на законодавчому рівні (вимога FISMA та FedRAMP);
- Фінансовий сектор і банки: міжнародні стандарти захисту платіжних систем (наприклад, PCI-DSS) вимагають використання сертифікованих криптографічних засобів або HSM для обробки банківських карток;
- Хмарні сервіси та B2B-продукти: сертифікація підтверджує для корпоративних клієнтів, що бібліотека не містить грубих помилок і пройшла незалежне тестування в акредитованій лабораторії.

1.8 Чому для реалізації RSA та деяких інших асиметричних криптосистем необхідна арифметика багаторозрядних чисел? Які вимоги, крім швидкодії, висуваються до її реалізації у криптографічному програмному забезпеченні?

Стандартний рівень стійкості RSA вимагає використання ключів довжиною від 2048 до 4096 біт [13]. Звичайні процесори вміють напряду працювати лише з числами розрядністю 32 або 64 біти. Тому для RSA потрібні спеціальні бібліотечні алгоритми «великих

чисел» (*BigNum*), які розбивають гігантські числа на масиви та виконують над ними модульне множення й піднесення до степеня [12]:

$$c \equiv m^e \pmod{N}, \quad m \equiv c^d \pmod{N} \quad (1)$$

Вимоги до реалізації (крім швидкодії):

- Захист від витоків за часом: модульне піднесення до степеня закритим ключем має виконуватися за однаковий час незалежно від кількості нулів чи одиниць у ключі (захист від часових атак);
- Криптографічне засліплення (Blinding): перемішування вхідного повідомлення з випадковим числом перед обчисленням закритим ключем ($m' = m \cdot r^e \pmod{N}$), щоб сторонній спостерігач не міг зіставити вхідні дані з часом або споживанням енергії [12];
- Безпечне очищення пам'яті (Zeroization): проміжні результати та фрагменти ключа в оперативній пам'яті мають надійно затиратися нулями після завершення операції, щоб компілятор не оптимізував це очищення (`explicit_bzero`);
- Контроль переповнення (Memory Safety): гарантована безпека коду від помилок роботи з пам'яттю (buffer overflow) під час розрахунку великих добутоків.

2 Порівняння криптографічних бібліотек

2.1 Обрані криптографічні бібліотеки

Для проведення порівняльного аналізу було обрано дві найпопулярніші криптографічні бібліотеки мови програмування Python:

- **cryptography** [5, 6] — офіційно рекомендована спільнотою Python Cryptographic Authority (PyCA) бібліотека. Вона побудована як надійна обгортка над перевіреними низькорівневими C-бібліотеками (насамперед OpenSSL) з безпечним внутрішнім ядром на мові Rust. Її архітектурна концепція чітко розмежовує простий високорівневий безпечний шар (безпечні рецепти типу **Fernet**) та низькорівневий шар примітивів **hazmat** (Hazardous Materials).
- **PyCryptodome** [14, 15] — самостійний форк занедбаної бібліотеки **PyCrypto**, створений для забезпечення зворотної сумісності, суттєвого розширення підтримуваних алгоритмів та виправлення критичних проблем безпеки й швидкодії. Вона є самодостатньою бібліотекою, що містить власні реалізації алгоритмів на мові C без жорсткої прив'язки до зовнішнього системного OpenSSL.

2.2 Порівняльний аналіз бібліотек

Критерій	cryptography [5] and [6]	PyCryptodome [14], [15] and [16]
Мови та платформи	Мова: Python 3.9+, PyPy 3.11 Платформи: CentOS Stream 9–10 (x86-64); Fedora (x86-64); macOS 26 Tahoe (ARM64); Ubuntu 22.04, 24.04, 26.04+ (x86-64, ARM64, ARMv7l, ppc64le); Debian 12 Bookworm, 13 Trixie, Sid (x86-64); Alpine (x86-64, ARM64); Windows Server 2025 (64-bit).	Мова: Python 3.8+, PyPy. Критичні операції реалізовані на C. Платформи: Linux, Windows, macOS; архітектури x86, x86-64, ARM (включаючи ARM64 / Apple Silicon). Поширюється у вигляді попередньо скомпільованих бінарних коліс (wheels).
Симетричне шифрування	Підтримується. Бібліотека містить реалізації симетричних алгоритмів, зокрема AES, ChaCha20, Camellia, 3DES та ін., а також різні режими роботи блочних шифрів.	Підтримується. Містить як сучасний AES (128, 192, 256 біт), Salsa20, ChaCha20, так і велику кількість застарілих/легасі-шифрів: DES, 3DES, Blowfish, CAST-128, RC2, ARC4. Підтримує режими ECB, CBC, CFB, OFB, CTR, OpenPGP.
AEAD	Підтримується. Є готові реалізації AES-GCM, ChaCha20-Poly1305, AES-CCM, AES-SIV, AES-GCM-SIV та AES-OCB3.	Підтримується. Реалізовано режими блокового автентифікованого шифрування GCM, CCM, EAX, SIV, OCB, а також комбінацію ChaCha20-Poly1305. Дозволяє передавати відкриті асоційовані дані (AAD).
Асиметрична криптографія	Підтримується. Бібліотека містить RSA, Diffie–Hellman, ECDH, ECDSA, Ed25519, Ed448, X25519, X448, а в актуальних версіях також постквантові ML-KEM та ML-DSA.	Підтримується. Реалізовано алгоритми RSA (розмір ключів 1024–4096+ біт, схеми PKCS#1 v1.5 та OAEP), DSA, а також еліптичні криві (ECC): стандартні криві NIST (P-256, P-384, P-521), Ed25519, X25519. Постквантові алгоритми відсутні.

Цифровий підпис	Підтримуються RSA-PSS, RSA-PKCS1v1.5, ECDSA, Ed25519, Ed448 та ML-DSA.	Підтримуються схеми RSA-PSS, RSA-PKCS1v1.5, DSA, ECDSA (згідно зі стандартом FIPS 186-4) та EdDSA (Ed25519).
Геш-функції та KDF	Підтримуються SHA-2, SHA-3, BLAKE2, SHA-1, MD5, SM3, а також XOF.	Підтримуються SHA-1, SHA-2, SHA-3 (включно з SHAKE128/256, Кессак), BLAKE2 (2b, 2s), MD5; MAC-функції: HMAC, CMAC, Poly1305; KDF: scrypt, PBKDF2, HKDF, bcrypt.
Ініціалізація генератора випадкових чисел	Не потребує ручного налаштування PRNG користувачем. Випадкові значення генеруються за допомогою захищених механізмів операційної системи та OpenSSL. Тому користувачеві не потрібно самостійно додавати «шум» для ініціалізації генератора.	Не потребує ручної ініціалізації та підмішування ентропійного «шуму» користувачем у просторі користувача. Модуль повністю покладається на системне джерело ентропії ОС.
Криптографічно стійкі випадкові значення	Підтримується. Для цього можна використовувати <code>os.urandom()</code> , а також функції та методи бібліотеки, які генерують ключі випадковим чином. Наприклад, <code>AESGCM.generate_key()</code> генерує випадковий ключ необхідної довжини.	Підтримується функцією <code>Crypto.Random.get_random_bytes(n)</code> , яка є криптографічно безпечною та звертається безпосередньо до системного генератора випадкових чисел ОС (<code>os.urandom / getrandom</code>).
Високорівневий API	Підтримується. Бібліотека має високорівневий API з готовими безпечними криптографічними конструкціями та низькорівневий <code>hazmat</code> API. Рекомендується використовувати <code>high-level</code> API, коли це можливо.	Практично відсутній. Бібліотека орієнтована на низькорівневе та середньорівневе API, де розробник повинен самостійно контролювати параметри алгоритмів, генерувати Nonce/IV правильної довжини та вручну перевіряти теги автентифікації.

Апаратне прискорення	Через OpenSSL та його апаратно оптимізовані реалізації.	Підтримується за наявності апаратних інструкцій процесора: AES-NI та CLMUL (для прискорення обчислень GHASH у режимі AES-GCM) для архітектур x86/x64, а також ARM Crypto Extensions. За їх відсутності перемикається на оптимізовані C-процедури.
FIPS-сумісність / сертифікація	Власної FIPS-сертифікації не має; FIPS може забезпечуватися через відповідний FIPS-валідований модуль OpenSSL.	Власної сертифікації FIPS 140-2 / FIPS 140-3 не має. Не використовує системний OpenSSL, тому не може працювати через FIPS Object Module або FIPS Provider.
Ліцензія	LICENSE.APACHE, LICENSE.BSD	Подвійна ліцензія: BSD 2-Clause та Public Domain (спадщина коду вихідного PyCrypto).
Вразливості та історія зламів	Має історію оприлюднених вразливостей, які регулярно виправляються шляхом оновлення бібліотеки та її залежностей.	Має історію виправлених вразливостей (наприклад, DoS-вразливість у реалізації KDF script — CVE-2018-15560, витоки через сайд-канали в RSA — CVE-2023-52323). Проєкт інтегрований у Google OSS-Fuzz для превентивного виявлення дефектів пам'яті в коді на C.
Підтримка спільнотою	Проєкт активно розвивається та підтримується спільнотою PyCA, регулярно виходять оновлення й виправлення вразливостей. Наприклад, версія 50.0.1 була випущена 25 серпня 2026 року.	Підтримується переважно одним ключовим мейнтейнером (Helder Eijs / Legrandin) за підтримки відкритої спільноти на GitHub. Не має прямого інституційного чи корпоративного фінансування. Останній стабільний реліз 3.23.0. випущений 17 травня 2025 року, 3.24.0 наразі перебуває у розробці.

Актуальність документації	Документація є актуальною та регулярно оновлюється. Доступні приклади, API-довідка та GitHub Issues.	Документація на платформі ReadTheDocs є актуальною, містить детальний опис примітивів та готові фрагменти коду для кожного режиму роботи. Регулярно оновлюється під час нових релізів бібліотеки.
---------------------------	--	---

2.3 Обґрунтування вибору бібліотеки

Для виконання практичної частини та подальшої роботи обрано бібліотеку `cryptography` [5, 6]. Основні причини вибору:

- Захист від типових помилок (Misuse-resistance): наявність готових високорівневих інтерфейсів (як `AESGCM` чи `Fernet`) дозволяє шифрувати дані без ризику забути додати перевірку цілісності або помилитися з генерацією nonce. У `PyCryptodome` всі ці низькорівневі параметри доводиться контролювати вручну [14].
- Безпека пам'яті (Memory Safety): частина внутрішніх модулів бібліотеки написана на мові Rust, що суттєво знижує ризик класичних вразливостей переповнення буфера чи помилок виділення пам'яті, властивих для суто C-коду.
- Штатний криптографічний стандарт для Python: бібліотека підтримується офіційною організацією PyCA, постійно оновлюється, використовує системний OpenSSL і дозволяє працювати у FIPS-режимі за потреби [6].
- Сучасний функціонал: бібліотека активно впроваджує нові стандарти, зокрема постквантові алгоритми (ML-KEM, ML-DSA), що робить її значно перспективнішою для довгострокових проєктів порівняно з `PyCryptodome`.

3 Практичне використання криптографічних бібліотек

3.1 Симетричне автентифіковане шифрування

Встановлення бібліотеки.

```

1 import os
2 from cryptography.hazmat.primitives.ciphers.aead import AESGCM
3 from cryptography.exceptions import InvalidTag

```

3.1.1 Генерація криптографічно стійкого ключа

```

1 key = AESGCM.generate_key(bit_length=256)
2 aesgcm = AESGCM(key)
3
4 message = b"Hello, my name is Mariia"

```

3.1.2 Шифрування тестового повідомлення

```

1 nonce = os.urandom(12)
2 ciphertext = aesgcm.encrypt(nonce, message, None)
3
4 print("Message:", message)
5 print("Key:", key.hex())
6 print("Nonce:", nonce.hex())
7 print("Ciphertext:", ciphertext.hex())

```

3.1.3 Розшифрування повідомлення

```

1 decrypted = aesgcm.decrypt(nonce, ciphertext, None)
2
3 print("\nDecrypted message:", decrypted)

```

3.1.4 Перевірка цілісності після зміни шифротексту

```

1 modified_ciphertext = bytearray(ciphertext)
2 modified_ciphertext[0] ^= 1
3 modified_ciphertext = bytes(modified_ciphertext)
4
5 print("\nModified ciphertext:", modified_ciphertext.hex())
6
7 try:
8     aesgcm.decrypt(nonce, modified_ciphertext, None)
9     print("Decryption successful")
10 except InvalidTag:
11     print("Error: authentication tag verification failed")

```

3.1.5 Вивід

У результаті виконання програми було згенеровано 256-бітний ключ, 12-байтовий попси та отримано шифротекст тестового повідомлення.

```

1 Message: b'Hello, my name is Mariia'
2 Key: fc5febdc3df5cd09ecb790e5651e7dd441783d988c1e643e58acc8db8f25d

```

```
3 Nonce: 9f72c8ca0d7e717102b1c728
4 Ciphertext: 1
    e256814693b724c81fb72b20cb1ee6bcae3db883a2a5fb36416e6af01e720ed626effe2d343afd2
5
6 Decrypted message: b'Hello, my name is Mariia'
7
8 Modified ciphertext: 1
    f256814693b724c81fb72b20cb1ee6bcae3db883a2a5fb36416e6af01e720ed626effe2d343afd2
9 Error: authentication tag verification failed
```

Після зміни одного байта шифротексту його розшифрування завершилося помилкою `InvalidTag`. Це свідчить про виявлення порушення цілісності зашифрованих даних.

3.1.6 Призначення ключа, nonce та authentication tag

У використаному алгоритмі AES-GCM ключ є секретним значенням, за допомогою якого виконується шифрування та розшифрування повідомлення. У цьому прикладі використовується ключ довжиною 256 біт.

Nonce — це значення, яке використовується разом із ключем під час шифрування. У нашому випадку nonce має довжину 12 байт (Для AES-GCM рекомендована довжина nonce становить 12 байт (96 біт)) і генерується за допомогою криптографічно стійкого генератора випадкових чисел. Для AES-GCM важливо, щоб nonce не повторювався при використанні одного й того самого ключа. Повторне використання nonce з тим самим ключем є небезпечним, оскільки може призвести до розкриття інформації про відкритий текст та порушення автентифікаційного захисту. Тому для кожного нового повідомлення необхідно використовувати новий nonce.

Authentication tag — це тег, який генерується під час шифрування та використовується для перевірки цілісності й автентичності шифротексту. У використаній бібліотеці він формується автоматично.

Після зміни одного байта шифротексту перевірка authentication tag не проходить, тому виникає `InvalidTag`. Це показує, що AES-GCM забезпечує не тільки конфіденційність даних, а й перевірку їх цілісності.

3.2 Цифровий підпис

Для практичної демонстрації створення та перевірки цифрового підпису використано алгоритм на основі скрученої кривої Едвардса — Ed25519 (схема EdDSA). Він забезпечує високу швидкодію, стійкість до атак за часом виконання та фіксовану довжину підпису (64 байти) і відкритого ключа (32 байти).

Імпорт необхідних модулів:

```
1 from cryptography.hazmat.primitives.asymmetric import ed25519
2 from cryptography.hazmat.primitives import serialization
3 from cryptography.exceptions import InvalidSignature
```

3.2.1 Генерація ключової пари

```
1 private_key = ed25519.Ed25519PrivateKey.generate()
2 public_key = private_key.public_key()
3
4 pub_bytes = public_key.public_bytes(
5     encoding=serialization.Encoding.Raw,
6     format=serialization.PublicFormat.Raw
7 )
8
9 message = b"Hello, my name is Mariia"
10
11 print("Private key: generated (held in memory, not exported)")
12 print("Public key (raw hex):", pub_bytes.hex())
13 print("Original message:", message)
```

3.2.2 Створення цифрового підпису

```
1 signature = private_key.sign(message)
2
3 print("\nSignature (hex):", signature.hex())
4 print("Signature length:", len(signature), "bytes")
```

3.2.3 Перевірка цифрового підпису

```
1 try:
2     public_key.verify(signature, message)
3     print("Verification result: Signature is VALID")
4 except InvalidSignature:
5     print("Verification result: Signature is INVALID")
```

3.2.4 Перевірка підпису після зміни повідомлення

```
1 modified_message = b"Hello, my name is Mariia!"
2
3 print("\nModified message:", modified_message)
4
5 try:
6     public_key.verify(signature, modified_message)
7     print("Verification result: Signature is VALID")
```

```
8 except InvalidSignature:
9     print("Verification result: Signature verification FAILED (InvalidSignature)")
```

3.2.5 Вивід

У результаті виконання коду було згенеровано пару ключів Ed25519, обчислено цифровий підпис вихідного повідомлення та успішно верифіковано його за допомогою відкритого ключа.

```
1 Private key: generated (held in memory, not exported)
2 Public key (raw hex): 25
   aed4306658c129d87a34fff7b6497f09f3c8e30a5f694f69912c5b1ff45b88
3 Original message: b'Hello, my name is Mariia'
4
5 Signature (hex): 9
   b304a2c75ea4b89fffab3bd995d40930b295f1d012d4b279aaab3906b560ea6a3edc889198a5a15
6   454d2bd50b86103e060d23edafcdad683ad5e48d3ccb44504
7 Signature length: 64 bytes
8 Verification result: Signature is VALID
9
10 Modified message: b'Hello, my name is Mariia!'
11 Verification result: Signature verification FAILED (InvalidSignature)
```

3.2.6 Призначення ключів та цифрового підпису

У криптосистемі Ed25519 закритий ключ є секретним і застосовується виключно для накладання цифрового підпису, тоді як відкритий ключ поширюється вільно та дозволяє будь-якій стороні перевірити справжність документа. Цифровий підпис фіксованого розміру (64 байти) підтверджує авторство відправника та незмінність даних. Модифікація навіть одного знака у повідомленні (додавання знака оклику) призвела до завершення перевірки винятком `InvalidSignature`, що підтверджує надійне виявлення спотворення підписаних даних.

4 Вибір криптографічної бібліотеки для різних ІТ-систем

4.1 Сценарій А. Backend-сервіс

- Надійність та перевіреність. Сервер працює з конфіденційними даними - тому це обов'язково.
- Підтримка сучасних криптографічних алгоритмів. Необхідність, коли мова йде про надійні алгоритми шифрування, гешування та роботи з ключами.

- Якісний генератор випадкових чисел. Криптографічна стійка випадковість необхідна для генерації ключів, salt, nonce.
- Захист від типових помилок або атак. Ризик повинен бути зменшений.
- Оновлення безпеки. Попри гарну схему, важливо регулярно оновлення, щоб швидко виправити виявлені вразливості.

4.2 Сценарій Б. Мобільний застосунок

- Надійне шифрування локальних даних. Якщо зломисник отримає доступ до файлів застосунку, то конфіденційна інформація повинна бути збережена.
- Низьке споживання пам'яті. Мобільні пристрої більш обмежені, аніж сервери.
- Енергоефективність. Надмірна кількість операцій може швидко зменшувати заряд мобільного пристрою.
- Безпечне очищення пам'яті. Після використання секретні дані бажано видалити з пам'яті.
- Безпечне зберігання ключів. Ключ не можна просто залишити у файлі або в коді застосунку.

4.3 Сценарій В. Сервіс цифрового підпису

- Підтримка стандарту PKCS#11 та взаємодії з HSM (Hardware Security Module). Оскільки закритий ключ є найціннішим активом, він ніколи не повинен потрапляти у відкритому вигляді в оперативну пам'ять сервера; бібліотека має вміти делегувати операцію підпису зовнішньому апаратному модулю безпеки.
- Захист від атак побічними каналами (Constant-Time Implementation). Реалізація алгоритмів підпису (RSA, ECDSA, Ed25519) повинна мати постійний час виконання та використовувати техніку криптографічного засліплення (*blinding*) для виключення витоку частин закритого ключа через вимірювання часу чи енергоспоживання.
- Стійкість до дефектів обчислень (Fault Injection Resistance). Під час використання оптимізації CRT для RSA бібліотека обов'язково повинна верифікувати підпис перед відправленням клієнту, запобігаючи витоку множників ключа через обчислювальні збої.
- Суворе відповідність нормативним стандартам і FIPS-сертифікація. Сервіси цифрового підпису зазвичай мають юридичну силу, тому криптографічний модуль повинен мати сертифікат відповідності стандартам (FIPS 140-3, eIDAS, Common Criteria).

Обраний засіб: Програмний інтерфейс на базі PKCS#11 (наприклад, *OpenSSL* із відповідним engine/provider або *SunPKCS11*) у зв'язці з апаратним модулем безпеки HSM.

4.4 Сценарій Г. Embedded/IoT-пристрій

- Мінімальний розмір коду (Small Footprint) та відсутність динамічного виділення пам'яті. Бібліотека повинна займати мінімальний обсяг ROM/Flash і RAM, дозволяти статичне виділення буферів та уникати фрагментації обмеженої динамічної пам'яті (heap).
- Підтримка апаратних криптографічних блоків мікроконтролера. Можливість задіяти вбудовані в кристал ESP32 апаратні прискорювачі AES, SHA та RSA/ECC для прискорення обчислень та збереження заряду акумулятора.
- Використання алгоритмів на еліптичних кривих (ECC). Застосування Ed25519 або ECDSA замість ресурсомісткого RSA, оскільки еліптична криптографія забезпечує високу стійкість за значно меншої довжини ключів (256 біт замість 2048 біт) і менших витрат оперативної пам'яті.
- Підтримка апаратного джерела ентропії (TRNG). Для безпечної генерації ключів та попси бібліотека повинна отримувати випадкові значення безпосередньо з апаратного генератора шуму ESP32, а не використовувати звичайні некриптографічні PRNG.

Обраний засіб: Бібліотека *Mbed TLS*, інтегрована до складу *ESP-IDF*, з увімкненою апаратною підтримкою мікроконтролера ESP32.

5 Індивідуальне завдання

У межах індивідуального завдання розглядається сценарій А. Backend-сервіс

5.1 Основні вимоги

Основні вимоги до криптографічного програмного забезпечення для цього сценарію були визначені в попередньому розділі. Далі на їх основі обирається конкретна криптографічна бібліотека та визначаються необхідні для Backend-сервісу криптографічні механізми.

5.2 Обрана криптографічна бібліотека

Для реалізації криптографічних операцій обрано бібліотеку `cryptography` для мови Python. Вона надає високорівневі інтерфейси для типових криптографічних операцій, а також низькорівневий API для спеціалізованих задач. Бібліотека підтримує симетричне та асиметричне шифрування, цифрові підписи, геш-функції та функції виведення ключів[5].

5.3 Необхідні криптографічні функції

Для захисту конфіденційних даних у Backend-сервісі доцільно використовувати симетричне шифрування з автентифікацією, зокрема AES-GCM. Такий режим забезпечує

не лише конфіденційність, а й контроль цілісності та автентичності зашифрованих даних. Для операцій з відкритими ключами можуть використовуватися асиметричні алгоритми, а для підтвердження автентичності повідомлень — цифрові підписи. Також необхідними є криптографічні геш-функції та функції виведення ключів[5].

5.4 Робота з ключовою інформацією

Оскільки сервер працює з конфіденційними даними, ключі не повинні зберігатися безпосередньо у вихідному коді застосунку. Для їх зберігання доцільно використовувати спеціалізовані засоби керування ключами та секретами. Генерація ключів, попсо та інших випадкових значень повинна здійснюватися за допомогою криптографічно стійкого генератора випадкових чисел. Крім того, необхідно забезпечити контроль доступу до ключів та їх своєчасну заміну відповідно до визначеного життєвого циклу [1].

5.5 Обґрунтування вибору бібліотеки

Вибір бібліотеки `cryptography` зумовлений тим, що вона охоплює основні криптографічні операції, необхідні для серверного застосунку, і водночас надає високорівневий API для типових задач. Це дозволяє не реалізовувати криптографічні алгоритми самостійно та зменшує ризик помилок під час їх використання. Наявність підтримки AES-GCM, асиметричної криптографії, цифрових підписів і функцій виведення ключів робить бібліотеку придатною для розглянутого сценарію [5].

5.6 Ризики неправильного використання

Основні ризики пов'язані не стільки з використанням самої бібліотеки, скільки з неправильним застосуванням її можливостей. Наприклад, повторне використання попсо з одним ключем під час роботи AES-GCM може призвести до порушення безпеки шифрування [5]. Також небезпечними є використання застарілих алгоритмів, слабких ключів, неправильне зберігання ключової інформації та некоректне використання низькорівневого API. Окремою проблемою є використання звичайних криптографічних геш-функцій для зберігання паролів замість спеціалізованих алгоритмів, призначених для цього завдання [1].

Висновки

Під час виконання лабораторної роботи ми дослідили теоретичні та практичні засади функціонування сучасних криптографічних бібліотек, критерії їх вибору для різних архітектурних платформ, а також здобули практичні навички роботи з криптографічними API.

Аналізуючи принципи побудови криптографічного ПЗ, ми з'ясували, що самостійна реалізація стандартних алгоритмів є неприпустимою в прикладному програмуванні,

оскільки навіть математично точний код без належного аудиту стає вразливим до атак побічними каналами (зокрема, через процесорний кеш та коливання часу виконання). Саме тому критично важливо використовувати готові й стандартизовані бібліотеки. Водночас застосування низькорівневих інтерфейсів (*hazmat*) вимагає глибокого розуміння примітивів, адже помилки в ручному виборі режимів шифрування, повторне використання попси або некоректна обробка доповнення нівелюють увесь захист. Натомість високорівневі API завдяки підходу *misuse-resistance* мінімізують людський фактор і гарантують безпеку за замовчуванням.

На основі детального зіставлення бібліотек `cryptography` та `PyCryptodome` за 15 параметрами ми визначили, що для екосистеми Python найбільш обґрунтованим вибором є `cryptography`. Ця бібліотека спирається на надійний бекенд OpenSSL у поєднанні з пам'ятобезпечним ядром на мові Rust, активно розвивається спільнотою PyCA, підтримує FIPS-сумісний режим роботи та забезпечує доступ до сучасних криптографічних стандартів.

Практична частина роботи дозволила нам на практиці переконатися в надійності сучасних криптографічних механізмів. Експеримент із симетричним автентифікованим шифруванням AES-GCM наочно підтвердив властивості цілісності AEAD-режимів: спотворення хоча б одного байта шифротексту миттєво викликало помилку автентифікаційного тегу (`InvalidTag`). Аналогічно, під час дослідження цифрового підпису Ed25519 будь-яка спроба модифікації вхідного повідомлення робила підпис недійсним із викликом винятку `InvalidSignature`, що підтверджує його чутливість до змін і здатність гарантувати невідомість авторства.

Розглядаючи вимоги до різних IT-систем, ми дійшли висновку, що криптографічні засоби мають суворо відповідати обмеженням середовища: для мобільних застосунків та сервісів підпису пріоритетом є апаратна ізоляція ключів (Keystore, Secure Enclave, HSM), тоді як для IoT-пристроїв ключовими залишаються компактність коду та апаратне прискорення. У межах індивідуального завдання для Backend-сервісу ми обґрунтували, що належний захист конфіденційних даних досягається лише комплексним підходом: поєднанням стійкого алгоритму AES-GCM, безпечного виведення ключів, відмови від збереження секретів у вихідному коді на користь спеціалізованих засобів керування ключами та обов'язкового використання системних генераторів випадкових чисел.

Перелік використаних джерел

Література

- [1] *OWASP*.
<https://cheatsheetseries.owasp.org/index.html>
- [2] *Cryptographic Primitive*.
https://csrc.nist.gov/glossary/term/cryptographic_primitive

- [3] *Crypto Protocols*.
https://web.archive.org/web/20170829004310/http://www.ccs-labs.org/~dressler/teaching/netzsicherheit-ws0304/07_CryptoProtocols_2on1.pdf
- [4] *Crypto Primitive Wiki*.
https://en.wikipedia.org/wiki/Cryptographic_primitive
- [5] *cryptography*.
<https://cryptography.io/en/stable/>
- [6] *cryptography git*.
<https://github.com/pyca/cryptography>
- [7] *Libsodium*.
<https://doc.libsodium.org/>
- [8] *Tink*.
<https://developers.google.com/tink>
- [9] *FIPS PUB 140-3*.
<https://csrc.nist.gov/pubs/fips/140-3/final>
- [10] *CMVP*.
<https://csrc.nist.gov/projects/cryptographic-module-validation-program>
- [11] *NIST SP 800-140*.
<https://csrc.nist.gov/pubs/sp/800/140/final>
- [12] *RFC 8017 (PKCS #1 v2.2)*.
<https://www.rfc-editor.org/rfc/rfc8017>
- [13] *NIST SP 800-57*.
<https://csrc.nist.gov/pubs/sp/800/57/pt1/r5/final>
- [14] *PyCryptodome*.
<https://pycryptodome.readthedocs.io/en/latest/>
- [15] *PyCryptodome git*.
<https://github.com/Legrandin/pycryptodome>
- [16] *NVD CVE-2023-52323 (PyCryptodome)*.
<https://nvd.nist.gov/vuln/detail/CVE-2023-52323>